

Analysis and Design of Health Insurance Policy and Claims Management System

Relational Database and Management Systems



The Problem: Manual Health Insurance Management

Duplicate Records

Same customer entered multiple times

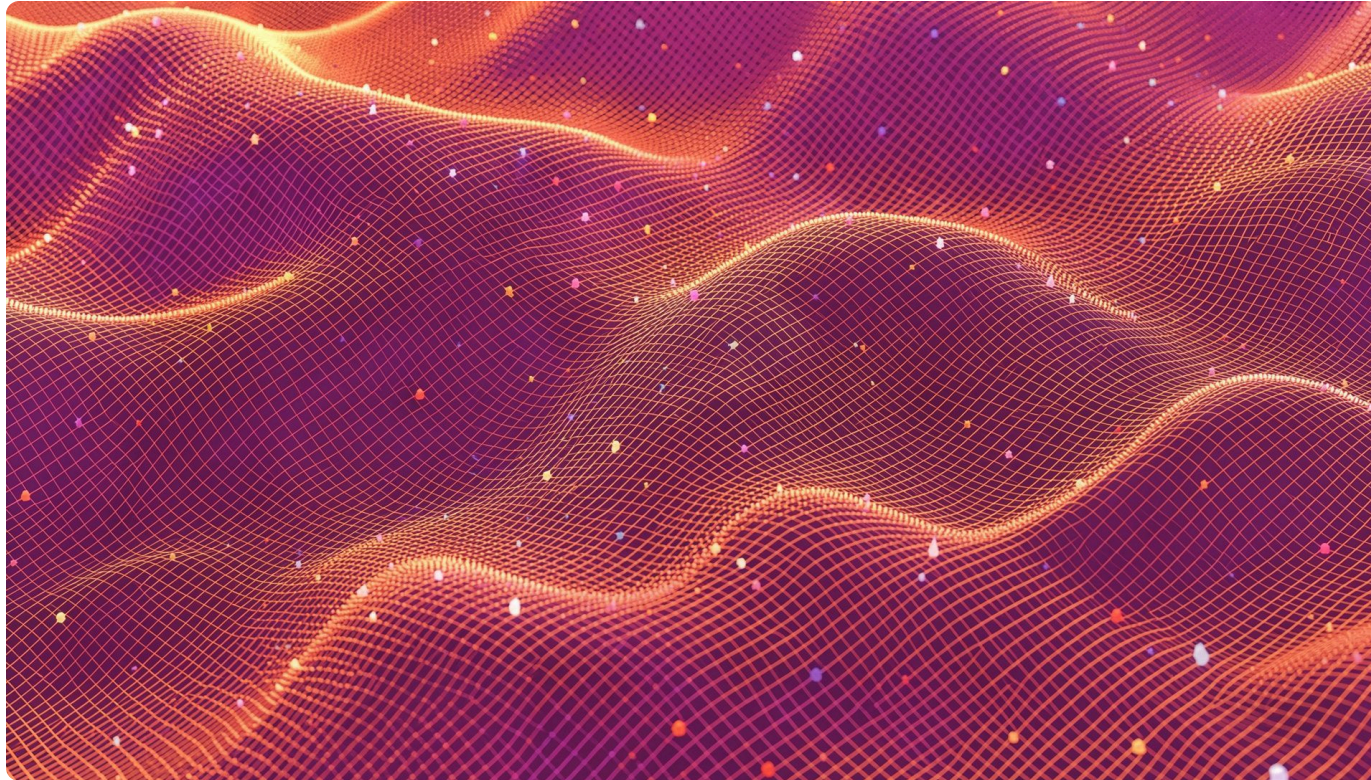
Slow Claim Processing

Manual tracking causes costly delays

Data Integrity Issues

Inconsistent information across systems

Real-World Impact



Time-taking Process

Claims take far too long to process manually

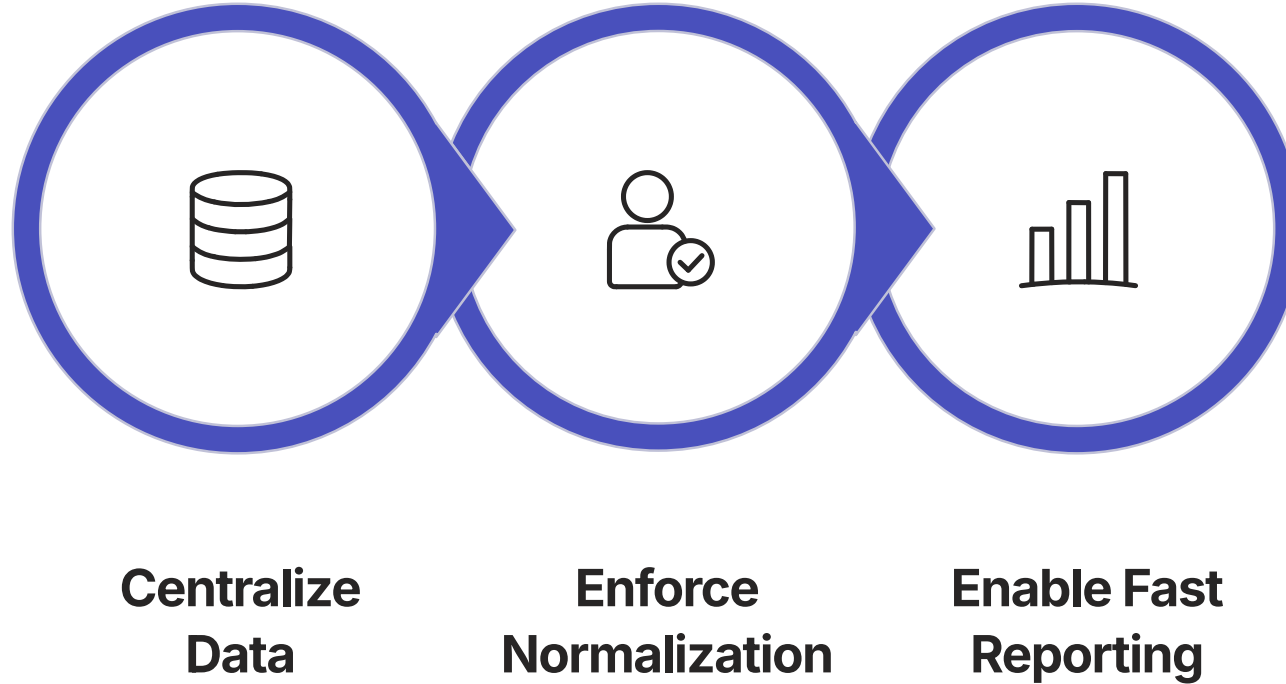
Repeated Customer Calls

No single source of truth forces customers to call again and again

No Quick Answers

"How many claims are pending?" — impossible to answer instantly

The Solution: Relational Database



A normalized relational database eliminates redundancy, enforces consistency, and enables automated reporting across all operations.

System Scope: 8 Core Entities



Customer

Policyholders



Policy Type

Coverage categories



Hospital

Network facilities



Treatment

Medical procedures



Policy

Insurance contracts



Insurance Company

Providers



Claim

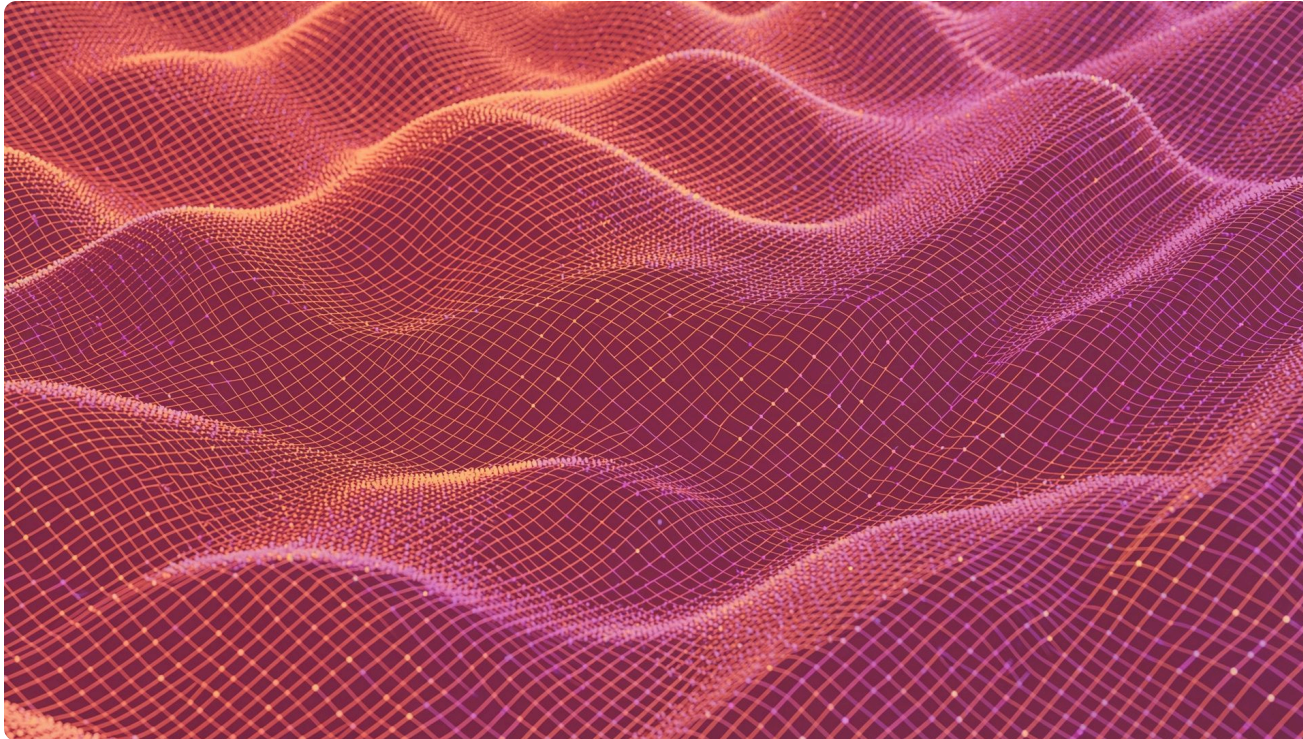
Submitted claims



Claim Payment

Approvals & payments

Entity: CUSTOMER



PRIMARY ENTITY

CUSTOMER Table

customer_id — Primary Key

Unique identifier for every policyholder

name, DOB, gender

Core demographic attributes

phone, email, address

Contact information — stored once, no duplication

Entity: POLICY

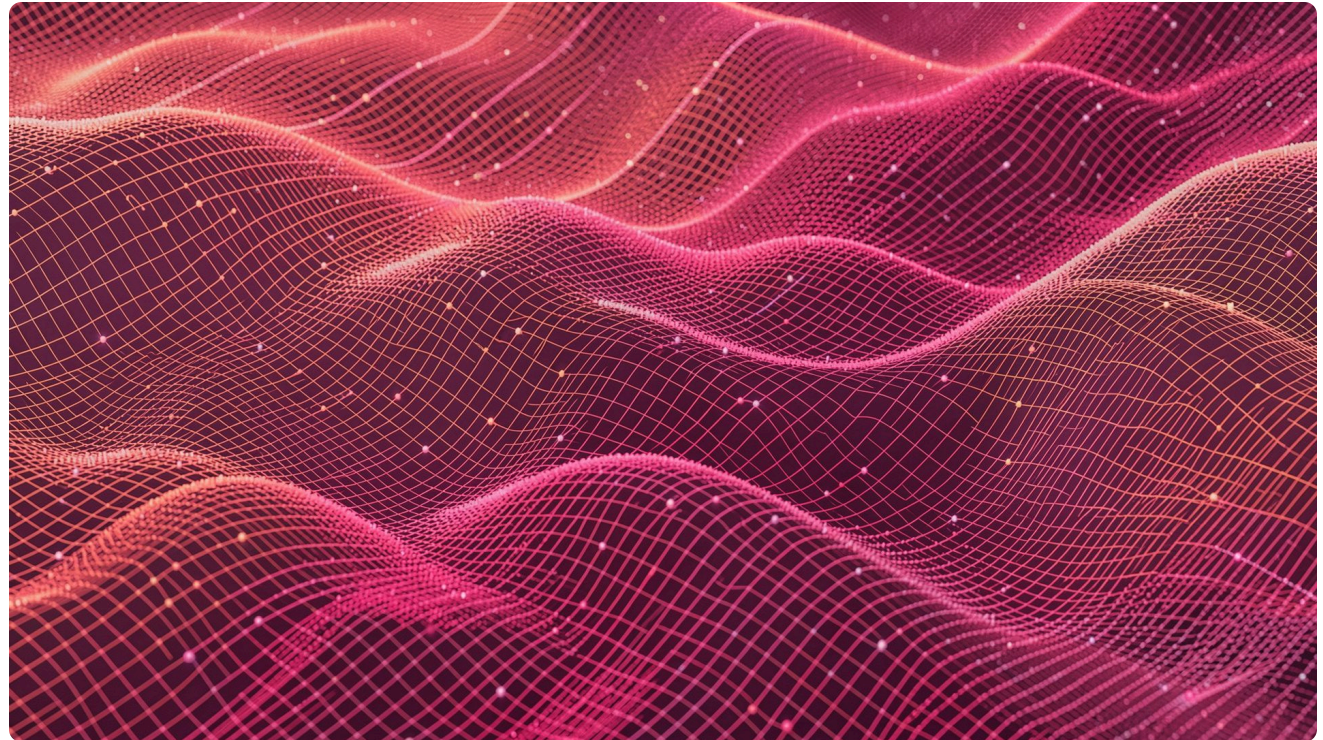
policy_id — Primary Key

customer_id — FK →
CUSTOMER

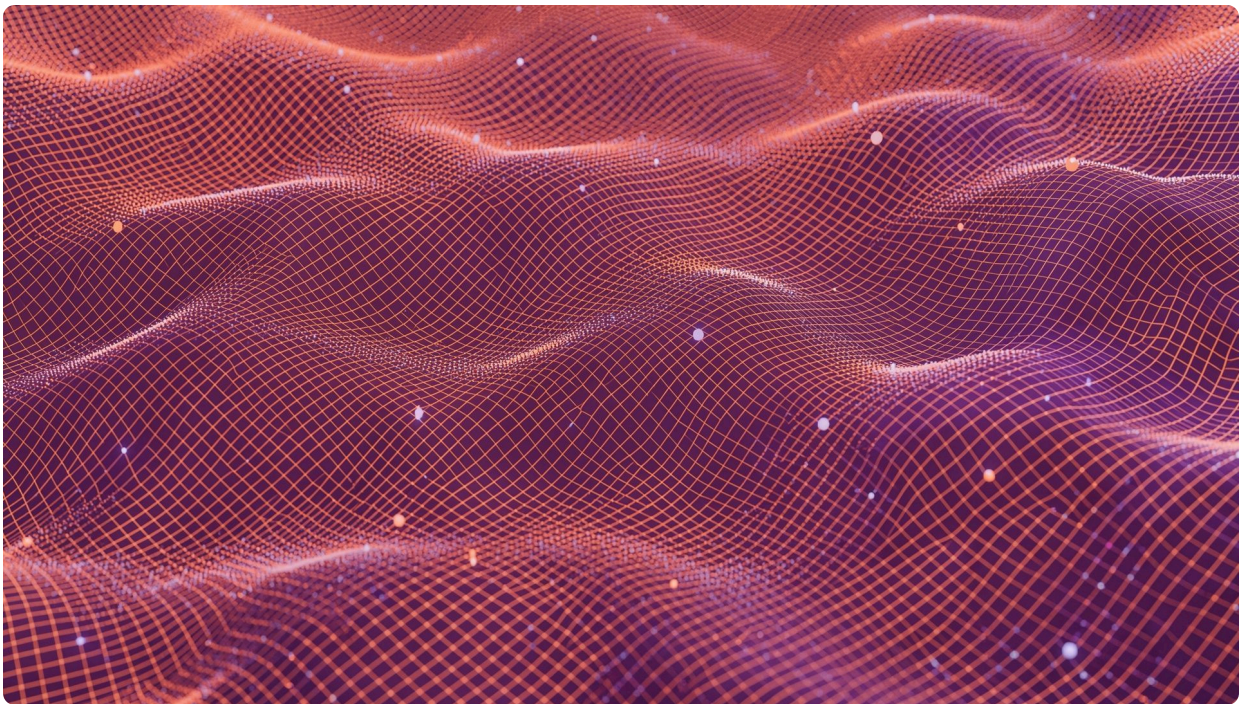
policy_type_id, company_id —
FK links

start_date, end_date, premium

**status: Active / Expired /
Cancelled**



Entity: CLAIM

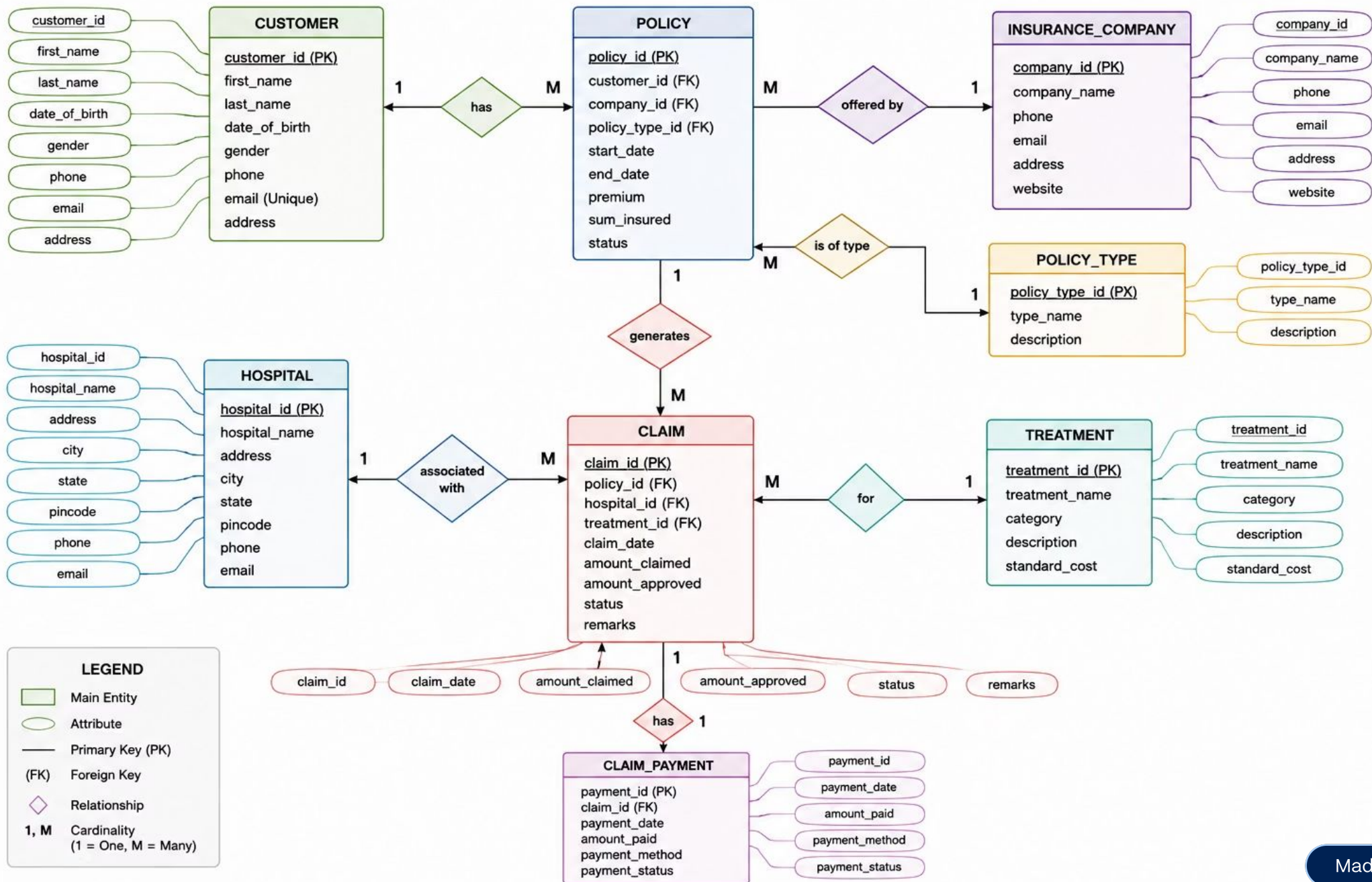


CENTRAL TRANSACTION ENTITY

CLAIM Table

Field	Role
claim_id	Primary Key
policy_id	FK → POLICY
hospital_id	FK → HOSPITAL
treatment_id	FK → TREATMENT
claim_date, amount_claimed, amount_approved	Financial details
status	Pending / Approved / Rejected

Relationships: The ER Diagram



Next Steps – Project Development

Move from the ER model into a working database system through a clear, sequential implementation plan.

01

Finalize Database Design

Review entities, relationships, and business rules to confirm the final ER structure.

02

Create Relational Schema

Convert the ER diagram into tables, keys, constraints, and relationships.

03

Implement in MySQL

Build the database objects in MySQL and apply the schema definition.

04

Populate Database

Insert sample and real data to make the system ready for testing and use.

05

Develop SQL Queries

Create queries for searching, reporting, filtering, and extracting insights.

06

Add Advanced Features

Extend the system with views, procedures, triggers, or other enhancements.

07

Generate Reports & Analysis

Summarize results and use the database output for decision-making and review.

Overall Flow: ER Design → Schema → MySQL Implementation → Data → Queries → Advanced Features → Reports